STUDENT IMPLEMENTATION

Student record and Academic Management System:-Data organization using python

Subject: Python

Student Name Satyam Pachpute

PRN: 2126UDSM1085

Objective:- Objective is to use Python built-in data structures to store, organize, update and retrieve Student academic information efficiently.

Problem Statement:- managing student academic records manually can be time-consuming and may lead to difficulties in storing, updating, searching, and retrieving information. Student details such as roll number, name, department, subjects, marks, attendance, and contact info need to be organized properly

Problem Understanding:- The Student Record and Academic Management System is designed to organize student academic information using suitable Python data structures. The system should store student information to be updated and retrieved and make it easier to manage academic records efficiently.

1. Student Information to Be Stored

- Student Name
- Roll Number
- Department
- Date of Birth
- Subjects
- Marks
- Attendance
- Contact information

2. Inputs

- Student Details
- Roll Number
- Department
- Subject
- Marks
- Attendance

- Contact Information

3. Outputs

- Student Records
- Search results
- Updated records
- Average marks
- Highest scorer

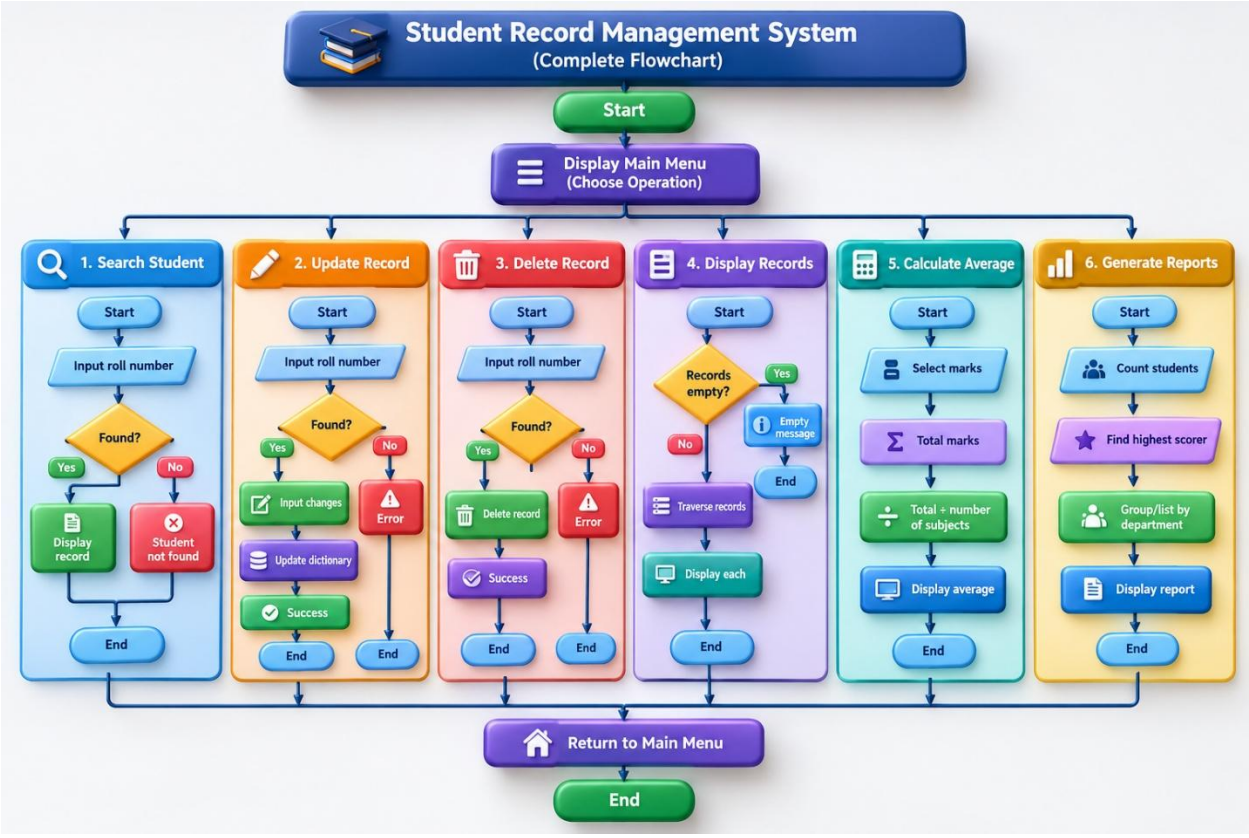
4. Relationship Between Data

A student record contains several types of information. The roll number identifies the student, while the name, department, subjects, marks, and contact information are associated with student.

5. Suitable Data Structures

- String: Name, Department, Addresses, E-mail.
- List: Student record, subject, marks, Attendance.
- Tuple: Roll no, Registration no, Date of Birth.
- Set: Department, subjects, student clubs.
- Dictionary: Complete Student Profile.

Flow chart :



ALGORITHM DESIGN

Algorithm 1: Search Student by Roll Number

Purpose: To search student by roll no.

1. Start
2. Enter the student's roll number
3. Search the student records
4. Compare the entered roll number with the stored numbers
5. If the roll number is found the student's complete information
6. otherwise, display **"Student record not found."**
7. Stop

Algorithm 2: Add a New Student Record

Purpose: To add new student information.

1. Start
2. Enter the student's roll no
3. Enter student's name
4. Enter registration no

5.Enter student's department

6.Enter DOB of student

7.Enter subjects

8.Enter marks of each subject

9.Enter attendance detail

10.Enter contact information

11.Store all the information in the student record

12.Display "**Student record added successfully."**

13.Stop

Algorithm 3: Update Student Information

Purpose: To modify an existing student's information.

1. Start

2. Enter the roll no of the student.

3. Search for the student using the roll no.

4. If student is found, display the existing information.

5.Select the information that needs to be updated.

6.Enter the new information.

7.Replace the old information with new information.

8.Display "**student record updated**."

9.If the roll no is not found, display "**student record not found**."

10.stop

Algorithm 4: Delete a Student Record

Purpose: To remove a student's record from the collection.

1. Start.

2. Ask the user to enter the roll number.

3. Search for the student.

4. If the student is not found, display **"Student not found"**.

5. If found, remove the student's record.

6. Display **"Record deleted successfully"**.

7. Stop.

Algorithm 5: Calculate Average Marks

1. Start.

3. Search for the student.

4. If the student is not found, display **"Student not found"**.

5. If found, retrieve the student's marks.
6. Add all the marks.
7. Count the number of subjects.
8. Calculate: sum of all marks/number of subjects
9.stop

Algorithm 6: Find the Highest Score

Purpose: To find the student with the highest marks.

1. Start.
2. Take the marks of all students.
3. Assume the first student's marks are the highest.
4. Compare them with the next student's marks.
5. If the next student's marks are higher, make them the new highest.
6. Continue until all students are checked.
7. Display the student with the highest marks.
8. Stop.

Algorithm 7: Count the Number of Students

Purpose: To find the total number of students.

1. Start.
2. Set a counter to 0.
3. Go through every student record.
4. For each student, increase the counter by 1.
5. After checking all records, display the counter.
6. Stop.

Algorithm 8: List Students by Department

Purpose: To display only students belonging to a particular department.

1. Start.
2. Ask the user to enter a department.
3. Go through all student records.
4. Compare each student's department with the entered department.
5. If they match, display that student's information.
6. Continue until all records are checked.
7. Stop.

PSEUDOCODE DEVELOPMENT

1.Pseudocode for Searching a Student

```
BEGIN

INPUT Roll Number

SEARCH for Roll Number in student records

IF Roll Number is found THEN
  DISPLAY complete student record
ELSE
  DISPLAY "Student not found"
END IF

END
```

2. Pseudocode for Updating Student Information

```
BEGIN

INPUT Roll Number

SEARCH for Roll Number in student records

IF Roll Number is found THEN
  INPUT new student information

  UPDATE the selected information
  DISPLAY "Record updated successfully"
ELSE
  DISPLAY "Student not found"
END IF
```

```
END
```

3.**Pseudocode for Deleting a Student Record**

```
BEGIN

INPUT Roll Number

SEARCH for Roll Number in student records

IF Roll Number is found THEN
  DELETE the student record
  DISPLAY "Record deleted successfully"
ELSE
  DISPLAY "Student not found"
END IF

END
```

4. Pseudocode for Displaying Student Records

```
BEGIN

IF student records are empty THEN
  DISPLAY "No student records available"
ELSE
  FOR each student record
    DISPLAY student information
  END FOR
END IF

END
```

5. Pseudocode for Calculating Average Marks

```
BEGIN
```

INPUT student record

SET Total Marks = 0
SET Number of Subjects = number of subjects

FOR each subject
  Total Marks = Total Marks + Marks of subject
END FOR

Average = Total Marks / Number of Subjects

DISPLAY Average

END

6. Pseudocode for Generating Reports

BEGIN

ACCESS all student records

CALCULATE average marks

FIND the student with highest marks

LIST students according to department

COUNT total number of students

ORGANIZE all information into a report

DISPLAY the generated report

END

7. Pseudocode for Adding a New Student

BEGIN

INPUT Roll Number

IF Roll Number already exists THEN
  DISPLAY "Duplicate record"
ELSE
  INPUT Name
  INPUT Department
  INPUT Subjects
  INPUT Marks
  INPUT Attendance
  INPUT Contact Information

  CREATE a student record
  STORE all student information

  ADD student record to the student collection

  DISPLAY "Student record added successfully"
END IF

END

PYTHON IMPLEMENTATION

1. String

```
name = "xyz"
department = "Computer Science"
address = "Aurangabad"
email = "xyz@gmail.com"
```

```
name = input("Enter student name: ")
department = input("Enter department: ")
print("name:", name)
print("Department:", department)
```

2. List

```
subjects = ["Python", "Maths", "Physics"]
marks = [80, 75, 90]
attendance = [85, 90, 80]


print("Subjects:", subjects)
print("Marks:", marks)
print("Attendance:", attendance)
```

3. Tuple

```
roll_number = (101,)
registration_number = ("REG2026CS101",)
date_of_birth = ("15-08-2007",)


print("Roll Number:", roll_number)
print("Registration Number:", registration_number)
print("Date of Birth:", date_of_birth)
```

4. Set

```
departments = {"Computer Science", "Mechanical", "Civil"}
subjects = {"Python", "Maths", "Physics"}
clubs = {"NSS", "Sports", "Coding Club"}


print("Departments:", departments)
print("Subjects:", subjects)
print("Student Clubs:", clubs)
```

5. Dictionary

```
student = {
    "roll number": 101,
    "name": "Rahul",
    "department": "Computer Science",
    "subjects": ["Python", "Maths", "Physics"],
```

```
        "marks": [80, 75, 90],
        "attendance": [85, 90, 80],
        "contact": "9876543210"
}


    print("Student Record")
    print("Roll Number:", student["roll number"])
    print("Name:", student["name"])
    print("Department:", student["department"])
    print("Subjects:", student["subjects"])
    print("Marks:", student["marks"])
    print("Attendance:", student["attendance"])
    print("Contact:", student["contact"])
```

DATA STRUCTURE INTEGRATION

It means **combining the Strings, Lists, Tuples, Sets, and Dictionaries you created into one Student Record Management System.**

```
    student = {
        "roll number": (101,),
        "name": "xyz",
        "department": "Computer Science",
        "subjects": ["Python", "Maths", "Physics"],
        "marks": [80, 75, 90],
        "attendance": [85, 90, 80],
        "contact": "9876543210"
}
```

TESTING REPORT

Test Case	Input	Expected Result
Add Student	Valid student details	Student added successfully
Search Student	Existing roll number	Student record displayed
Search Student	Non-existing roll number	"Student not found"
Delete Student	Existing roll number	Record deleted
Calculate Average	Valid marks	Average displayed
Duplicate Record	Existing roll number	Duplicate record rejected
Empty Record	No student records	"No records available"

OBSERVATIONS

1. The Student Record and Academic Management System successfully stores student information in an organized manner.
2. Different Python data structures are used according to the requirement:
    - Strings for names, departments, addresses, and email IDs.
    - Lists for subjects, marks, attendance, and student records.
    - Tuples for roll numbers, registration numbers, and dates of birth.
    - Sets for departments, subjects, and student clubs.
    - Dictionaries for complete student profiles.
3. The system can perform important operations such as adding, searching, updating, deleting, and displaying student records.
4. The system can calculate average marks and identify the highest scorer.
5. Department-wise student information and total student count can be generated as part of the report.
6. Testing helped identify invalid inputs, duplicate records, empty records, and other possible errors.
7. The integration of multiple Python data structures makes the system organized and suitable for managing student academic information.
8. The project demonstrates how Python collections can be combined to solve a real-world student record management problem.